Armé el TP5 siguiendo la estructura de carpetas del PDF: `index.html`, `css/tokens.css`, `data/planes.json` y `js/` con `estado`, `render`, `validaciones`, `cotizador`, `pago` y `app`, más un `README.md`.

Probé que la sintaxis de todos los módulos JS es correcta y que `calcularCuota()` da exactamente los 3 casos de prueba del PDF (2.000/3.000/4.500, 2.200/3.300/4.950 y 5.000/7.500/11.250). No pude abrir el sitio en un navegador, así que no revisé cómo se ve, ni la comparación con el Figma, ni el flujo completo ni los 390 y 1440 px.

Para comparar con el tuyo, acá van las decisiones que tomé:

- **Estado:** un único objeto `solicitud`, con un campo extra `error` para el motivo del pago. `render()` llena inputs, planes, stepper y resumen desde ese estado.
- **Campos:** cada input tiene un atributo `data-bind` con su ruta en el estado (por ejemplo `titular.email`). Un solo handler maneja `input` y `blur` para todos los campos.
- **Validaciones:** son funciones puras con los mensajes exactos del PDF. Cada botón deshabilitado muestra un texto con cuántos campos faltan.
- **Pago:** "Pagar" se deshabilita mientras procesa (1,5 s). Con la tarjeta 4000 0000 0000 0002 va a la pantalla de error. "Reintentar" vuelve al paso 4 con los datos, y "Cambiar medio de pago" además borra los datos de la tarjeta.
- **Responsive:** el resumen es una barra fija abajo en mobile y una card lateral en desktop (`lg:grid-cols-[1fr_380px]`).
- **Seguridad:** escapé los textos del usuario antes de ponerlos en el resumen.

Para probarlo, no lo abras con doble click: el `fetch` del JSON falla con file://. Usá `npx serve` o Live Server.

No incluí ningún bonus (localStorage, máscaras de input, modo oscuro, etc.). Tampoco están los links de "Editar" desde el resumen. En el README quedan pendientes la URL publicada, las capturas y la declaración de uso de IA, que tenés que completar vos.

Si querés, pasame el tuyo o contame en qué difiere, y lo comparo punto por punto contra el checklist de la sección 13.